ReactJS

Unit 2 — JSX & Components

JSX · Function components · Props · Nesting · children

In This Unit

1. What is JSX?
2. JSX rules and expressions
3. Components and props
4. Nesting components
5. Props children
6. Components in files

JSX

JavaScript XML · HTML in JavaScript

What is JSX?

- JSX = JavaScript XML
- It lets you write HTML-like code inside JavaScript
- JSX makes React code easier to read and write
- JSX is not valid JavaScript; a tool (Babel) converts it

JSX vs Without JSX

With JSX

```
const myElement =
  <h1>Hello JSX!</h1>;
```

Without JSX

```
const myElement =
  React.createElement(
    'h1', {}, 'No JSX!');
```

- JSX is cleaner and easier to understand
- Babel automatically converts JSX to createElement()

JSX Rule 1: Close All Tags

HTML

```
<p>Hello</p>        ✓ Correct
```

Wrong

```
<p>Hello            ✗ Error
```

Self-closing

```
<br />              ✓ Must close
```

- JSX follows XML rules: every tag must be closed
- Self-closing tags use /> notation
- Forgetting to close a tag throws an error

JSX Rule 2: className Instead of class

Correct: className

```
<h1 className="title">Hello</h1>
```

Wrong: class is reserved

```
<h1 class="title">Hello</h1>
```

- Use className instead of class
- class is a reserved word in JavaScript
- Babel converts className to class in the HTML

JSX Rule 3: Expressions with {}

```
const name = "Ravi";
const age = 20;

const msg = (
  <p>
    {name} is {age} years old
  </p>
);
```

Math

```
const sum = (
  <p>{5 + 5}</p>
);
```

Use {} to run JavaScript inside JSX.

JSX Rule 4: Inline Style Object

```
const myStyle = {
  color: "red",
  fontSize: "20px"
};

<h1 style={myStyle}>
  Styled text
</h1>
```

- Style is a JavaScript object, not a string
- CSS property names must be camelCase
- fontSize, not font-size

JSX Comments

```
{/* This is a comment */}

<p>
  Text
  {/* Comment inside JSX */}
</p>
```

Comments in JSX use {/ */} syntax, not //.*

COMPONENTS

Reusable pieces · Built from functions

What is a Component?

- A component is a reusable piece of the page
- Built using a JavaScript function
- Returns HTML (JSX) that will be shown

```
function Welcome() {
  return <h1>Welcome!</h1>;
}
```

Function Component: Basic Example

```
function Greeting() {
  return <h1>Hello React!</h1>;
}

const root =
  createRoot(document.getElementById('root'));
root.render(<Greeting />);
```

Result



Component Name Rules

- Component names must start with a capital letter (Welcome, Card, App)
- This tells React it is a component, not a regular HTML tag
- HTML tags are lowercase: , <p>, <h1>

✓ Correct	✗ Wrong
<Welcome />	<welcome />
function Car() {}	function car() {}

PROPS

Passing data to components

What are Props?

- Props are properties (data) sent to a component
- Like function arguments or HTML attributes
- Props are read-only (you cannot change them)

Passing props like HTML attributes

```
<Car brand="Ford" color="blue" />
```

Receiving Props: Code

```
function Car(props) {
  return (
    <h2>
      I am a {props.color} {props.brand}
    </h2>
  );
}

<Car brand="Ford" color="red" />
```

Result

I am a red Ford

Props Can Be Any Type

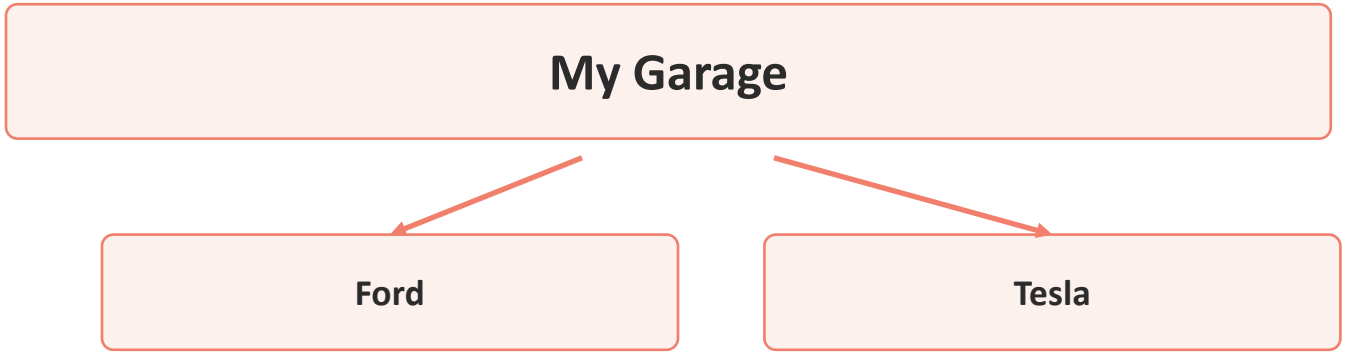
- String: brand="Ford"
- Number: age={20} (use {})
- Object: info={{name: 'Ravi', age: 20}}
- Array: skills={['React', 'JS']}

Nesting: Components Inside Components

```
function Car(props) {
  return <p>{props.brand}</p>;
}

function Garage() {
  return (
    <div>
      <h1>My Garage</h1>
      <Car brand="Ford" />
      <Car brand="Tesla" />
    </div>
  );
}
```

Result of Nesting



Garage component renders both Car components

Props Children

- children is a special prop that holds content between opening and closing tags

Parent

```
<Card>
  <p>This is inside the Card</p>
</Card>
```

Child component

```
function Card(props) {
  return (
    <div>
      {props.children}
    </div>
  );
}
```

props.children lets you pass JSX as content.

Export and Import Components

Car.jsx (separate file)

```
// Car.jsx
export function Car(props) {
  return <h2>{props.brand}</h2>;
}
```

App.jsx (imports Car)

```
// App.jsx
import { Car } from './Car';

function App() {
  return <Car brand="Ford" />;
}
```

Keep each component in its own file for reusability.

CLASS COMPONENTS

Older style · Still supported

Class Components

- Components can be written as JavaScript classes
- Class components extend React.Component
- They have a render() method that returns JSX
- Older React code uses class components
- Function components with Hooks are now preferred

Basic Class Component

```
import React from 'react';

class Greeting
  extends React.Component {
  render() {
    return <h1>Hello React!</h1>;
  }
}

export default Greeting;
```

Result



Props in Class Components

```
class Car extends React.Component {
  render() {
    return (
      <h2>
        I am a {this.props.color}
        {this.props.brand}
      </h2>
    );
  }
}

<Car brand="Ford" color="red" />
```

- Use this.props to access props
- Not just props like in function components

Class vs Function Components

Feature	Class	Function
Syntax	class X extends React.Component	function X() {}
Props	this.props.x	props.x
Render	render() method	return JSX
State	Older way	useState Hook
Preferred?	No	Yes

When to Use Each

Function Components

Modern choice. Use with Hooks for state and effects.

Class Components

Older code, some teams still use. Good to recognize.

For new projects, prefer function components with Hooks.

Practice

- Create a simple component called Greeting that returns a heading
- Add a name prop and use it in the heading
- Create a Card component that uses props.children
- Render the Greeting inside the Card
- Export both to separate files and import them in App

Summary

- JSX lets you write HTML inside JavaScript
- JSX rules: close tags, use className, expressions with {}
- Components are reusable functions that return JSX
- Pass data to components using props
- Nest components to build complex UIs
- Use props.children to pass content between tags